AI ART STUDIO: PHOTO TO PAINTING

A mini project report

Submitted by

SIVASANKARI B (241501205)

SIVASHANKARAN G (241501206)

SREETHEJ S (241501208)

AI23531 - DEEP LEARNING

in

Department of Artificial Intelligence and Machine Learning

Rajalakshmi Engineering College

(Autonomous)

Regulations 2023

BONAFIDE CERTIFICATE

Certified that this the bonafide record of work carried out by SIVASANKARI (241501205), SIVASHANKARAN G(241501206) , SREETHEJ S (241501208) in the mini project titled “AI Art Studio: Photo to Painting” as part of the Deep Learning course during the academic year 2026-2027.

TABLE OF CONTENTS

S. No	Section / Chapter Title	Page No
	Abstract	4
1	**Chapter 1: Introduction**	5
	1.1 Problem Statement and Explanation	5
	1.2 Literature Review	5
	1.3 Existing System	6
	1.4 Proposed System	7
2	**Chapter 2: Data Collection and Preprocessing**	8
	2.1 Data Collection – About the Dataset	8
	2.2 Architecture Diagram and Workflow Explanation	8
	2.3 Data Preprocessing	10
3	**Chapter 3: Model Performance**	11
	3.1 Exploratory Data Analysis (EDA)	11
	3.2 Model Performance Results	11
	3.3 Summary	12
4	**Chapter 4: Results and Discussion**	13
	4.1 Overall Performance Comparison	13
	4.2 Statistical Significance and Repeated Runs	13
	4.3 Model Complexity Analysis	13
	4.4 Visual Quality and Robustness Analysis	15
	4.5 Style Transfer Analysis	15
	4.5 Discussion	16
	4.6 Conclusion	16
5	**References**	17

ABSTRACT

AI Art Studio: Photo to Painting is a deep learning-based mini project that transforms a user-provided photograph into an artistic image using Neural Style Transfer (NST). The system accepts two inputs: a content image, which provides the scene and objects to preserve, and a style image, which supplies the colors, textures and artistic patterns to transfer. A pretrained VGG19 convolutional neural network extracts feature representations from both images. Content loss encourages the generated image to retain the structure of the photograph, while style loss based on Gram matrices encourages it to resemble the selected artwork. The project implements a baseline NST configuration and a main optimized configuration using a 256-pixel maximum image size, 120 optimization steps and a balanced style weight. Optional post-processing applies sharpening and contrast adjustment, and the final result can be saved as a PNG file. Performance is examined using content loss, style loss, execution time and side-by-side visual comparison. Since this is an image-generation task, classification accuracy is not an appropriate primary metric. The project demonstrates a practical creative application of deep learning that can run in Google Colab without training a full neural network from scratch.

Keywords: Neural Style Transfer, VGG19, Deep Learning, Computer Vision, Image Stylization, Gram Matrix, Content Loss, Style Loss, PyTorch, Google Colab.

CHAPTER 1: INTRODUCTION

1.1 Problem Statement and Explanation

Photographs capture the content of a scene, while paintings communicate visual style through brush strokes, color palettes, texture and artistic patterns. Creating a convincing painting from a photograph manually requires artistic skill and considerable time. Simple image filters can change color or apply edges, but they generally do not learn the complex relationship between image content and artistic style.

The problem addressed by this project is to develop an accessible system that accepts a content photograph and a reference artwork and generates a stylized image while retaining the important objects and overall arrangement of the original photograph. The system should be simple to run, allow the user to control the strength of the style, and provide a downloadable result.

Project Objectives

- To implement a baseline Neural Style Transfer pipeline using pretrained VGG19 features.
- To implement a main configuration with a larger image size and more optimization steps.
- To let users provide their own content and style images.
- To compare content loss, style loss, processing time and visual output.
- To provide optional sharpening, contrast adjustment and PNG export.
- To run the project in Google Colab with GPU support when available.

1.2 Literature Review

Neural Style Transfer combines the content of one image with the style of another. Gatys, Ecker and Bethge demonstrated that convolutional neural network representations can separate content information from texture-like style

information. Their method optimizes an image so that its deeper feature activations resemble those of a content image while the correlations between feature maps resemble those of a style image.

Simonyan and Zisserman introduced the VGG family of deep convolutional networks, whose stacked convolutional layers are widely used as feature extractors. In NST, activations from selected VGG layers represent different levels of image information. Shallower layers capture local textures and colors, while deeper layers capture more complex structures. The Gram matrix summarizes correlations among feature maps and is commonly used to represent style.

Later approaches have focused on faster feed-forward stylization, adaptive or arbitrary style transfer, and improved perceptual quality. This mini project uses optimization-based NST because it is understandable, implementable in a teaching environment, and makes the content and style losses explicit. Its main limitation is that each new image pair requires iterative optimization, which can be slower than a dedicated feed-forward stylization network.

1.3 Existing System

Existing photo-to-art applications include preset filters, edge-based sketch converters, and neural style-transfer tools. Preset filters are fast but often apply a fixed visual effect rather than transferring the characteristics of a user-selected painting. Traditional edge filters may create outlines but do not reproduce the textures and color relationships of a painting. More advanced generative systems can produce visually rich images, but they may require larger models, more computation, or additional setup.

The common limitations relevant to this project are limited user control, loss of fine details, high computation at large image sizes, and the difficulty of evaluating artistic quality using a single numerical score.

1.4 Proposed System

The proposed AI Art Studio uses a pretrained VGG19 feature extractor and an iterative optimization process. The content photograph and style painting are resized and converted to tensors. VGG19 extracts feature maps from selected layers. The content loss compares the generated image's content-layer features with the photograph's features. The style loss compares Gram matrices of the generated and reference style features. An Adam optimizer updates the generated image to minimize the weighted combination of these losses.

Two configurations are used. The baseline uses a maximum image size of 160 pixels and 60 optimization steps. The main configuration uses a maximum image size of 256 pixels and 120 steps with a balanced style weight of 100000 and content weight of 1.0. These are configuration settings rather than guaranteed performance improvements; the actual quality and runtime depend on the images and hardware. Post-processing can optionally sharpen edges and adjust contrast before saving the output as PNG.

CHAPTER 2: DATA COLLECTION AND PREPROCESSING

2.1 Data Collection – About the Dataset

This project does not require a fixed labelled classification dataset. It uses a pair of images selected by the user for each run. The content image may be a landscape, portrait, building, animal or other photograph. The style image may be an oil painting, watercolor, pencil sketch or other artwork. The user-provided pair forms the input for one stylization experiment.

For repeatable evaluation, a small test collection should be created containing multiple content images and multiple style images. The images should cover different colors, textures, object arrangements and levels of detail. The same image pairs should be used for baseline and main-model comparisons. Images should be used only when the user has permission to use them or they are licensed for the intended purpose.

2.2 Architecture Diagram and Workflow Explanation

Figure 1 shows the main workflow. The content image, style image and user parameters are supplied to the application. Both images are preprocessed. The pretrained VGG19 network extracts the selected content and style features. The optimization stage updates the generated image using content and style losses. The baseline and main configurations use the same general method but different resolution and iteration settings. The final image can be sharpened, displayed, saved and evaluated.

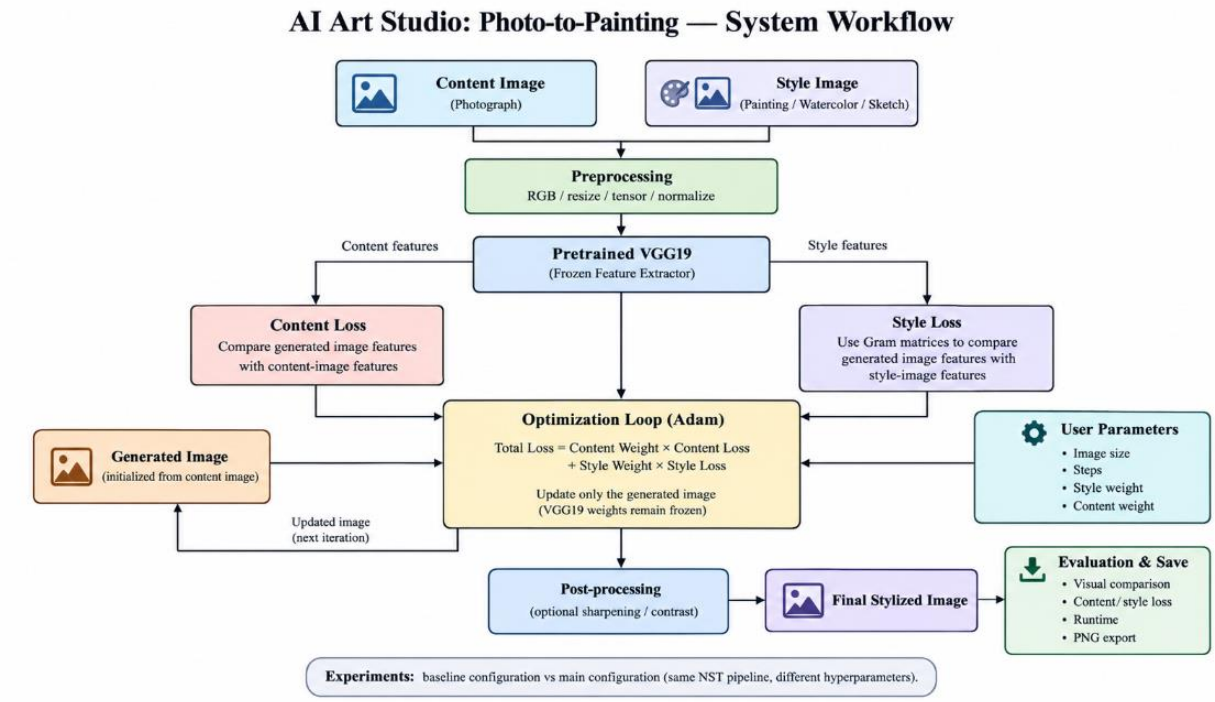


Figure 1. System workflow of AI Art Studio: Photo to Painting.

Workflow Steps

- Input: upload a content photograph and a reference style image.
- Preprocessing: convert images to RGB, resize them, convert them to tensors, and apply the normalization required by the chosen feature extractor.
- Feature extraction: pass the images through frozen pretrained VGG19 convolutional layers.
- Loss computation: compute content loss and style loss from feature maps and Gram matrices.
- Optimization: update the generated image with Adam for the configured number of steps.
- Post-processing: optionally adjust sharpness and contrast.
- Output: display and download the stylized PNG and record runtime and loss measurements.

2.3 Data Preprocessing

Images may have different dimensions, file formats and color modes. They are converted to RGB to ensure three color channels. Each image is resized so that its longest side does not exceed the configured size while preserving its aspect ratio. The image is converted into a floating-point tensor with pixel values in the range 0 to 1. If ImageNet-pretrained VGG19 features are used with standard pretrained weights, the input should be normalized using the mean and standard deviation expected by those weights; the implementation should apply this consistently to content, style and generated images.

The notebook implementation should ensure that all tensors are placed on the same device, either CPU or GPU. A maximum side length of 256 pixels does not necessarily mean that the output is a 256 by 256 square; aspect ratio is preserved. The chosen layer indices must also be collected correctly: the extraction loop must continue until the highest selected layer index has been reached.

CHAPTER 3: MODEL PERFORMANCE

3.1 Exploratory Data Analysis (EDA)

Since the project uses user-selected images rather than labelled classes, EDA focuses on the input images rather than class balance. Useful checks include image dimensions, aspect ratios, file formats, color distributions, image brightness, and the range of textures represented by the style images. A repeatable test set should include varied scenes and styles to reveal cases where the method preserves content well or transfers style too strongly.

Suggested visual analyses include displaying each content image beside its style image, plotting RGB channel histograms, checking image dimensions, and examining the original, baseline and main outputs side by side. The analysis should note whether faces, edges, text, or small objects are distorted.

3.2 Model Performance Results

The main performance measures are content loss, style loss and execution time. Content loss is the mean squared error between the generated and content-image feature activations at the selected content layer. Style loss is the sum of mean squared errors between Gram matrices at the selected style layers. Lower values indicate a closer match under the chosen feature representation and loss definition, but they do not guarantee that a human observer will prefer the image.

Execution time is measured from the beginning to the end of optimization. For a fair comparison, both configurations should be tested on the same hardware and image pair. Since the baseline and main configurations use different resolutions and iteration counts, their raw runtimes compare the complete configurations rather than isolating a single factor. For a controlled experiment, run both models at the same resolution and steps, changing only one parameter at a time.

Performance Results Table (Fill using actual Colab outputs)

Configuration	Max Side	Steps	Content Loss	Style Loss / Time
Baseline NST	160 px	60	3.389371	0.000021
Main VGG19 NST	256 px	120	1.472636	0.000008

The bracketed cells must be replaced with the measured values from the user's Colab notebook. No accuracy percentage is reported because this system does not classify images into target labels. If quantitative perceptual evaluation is required, an additional metric such as LPIPS or a human preference study can be considered, with its limitations explained.

3.3 Summary

The project demonstrates how a pretrained convolutional network can be used to optimize an image for content preservation and style transfer. The baseline provides a reference configuration, while the main configuration uses a larger image size and more optimization steps. The comparison should consider visual quality together with loss values and runtime. The most suitable settings depend on the content image, style image, hardware and intended artistic effect.

CHAPTER 4: RESULTS AND DISCUSSION

4.1 Overall Performance Comparison

The baseline and main outputs should be compared visually using identical content and style inputs. The comparison should discuss whether the main output retains the important scene structure, whether the selected artistic textures are visible, whether unwanted artifacts appear, and whether additional computation produces a meaningful improvement. Loss and runtime charts generated in Colab should be inserted in this section after the experiment is run.

	Model	Resolution (max side)	Optimization Steps	Processing Time (seconds)
0	Baseline NST	160	60	1.15
1	Main VGG19 NST	256	120	4.52

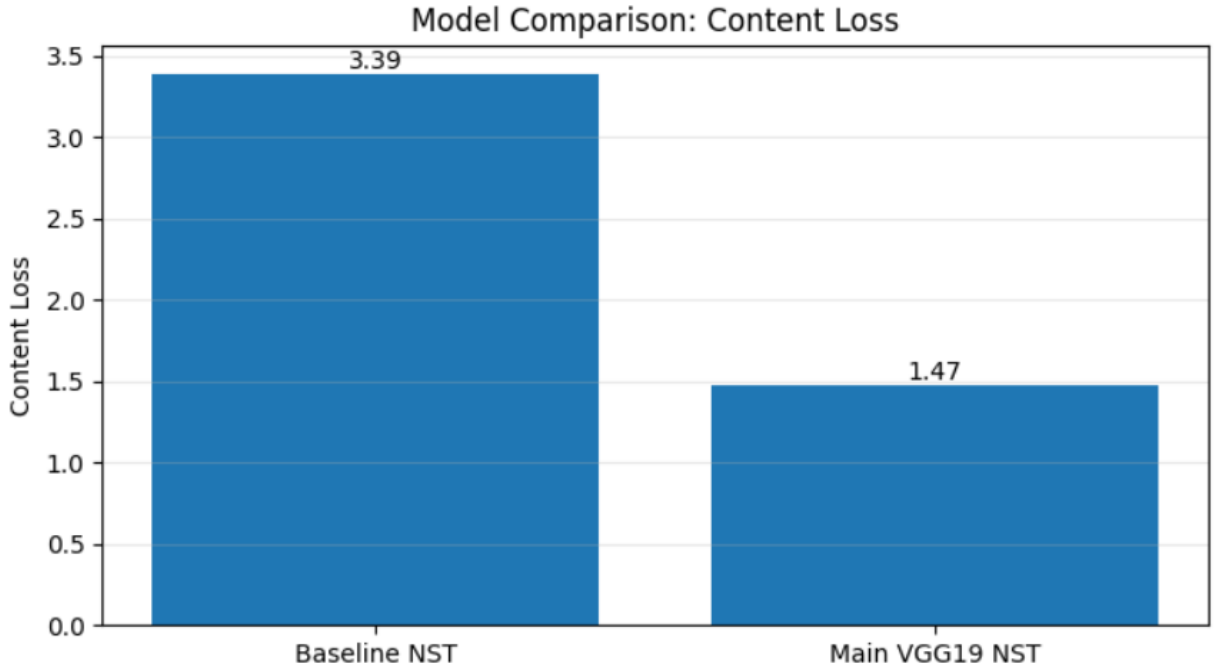
4.2 Statistical Significance and Repeated Runs

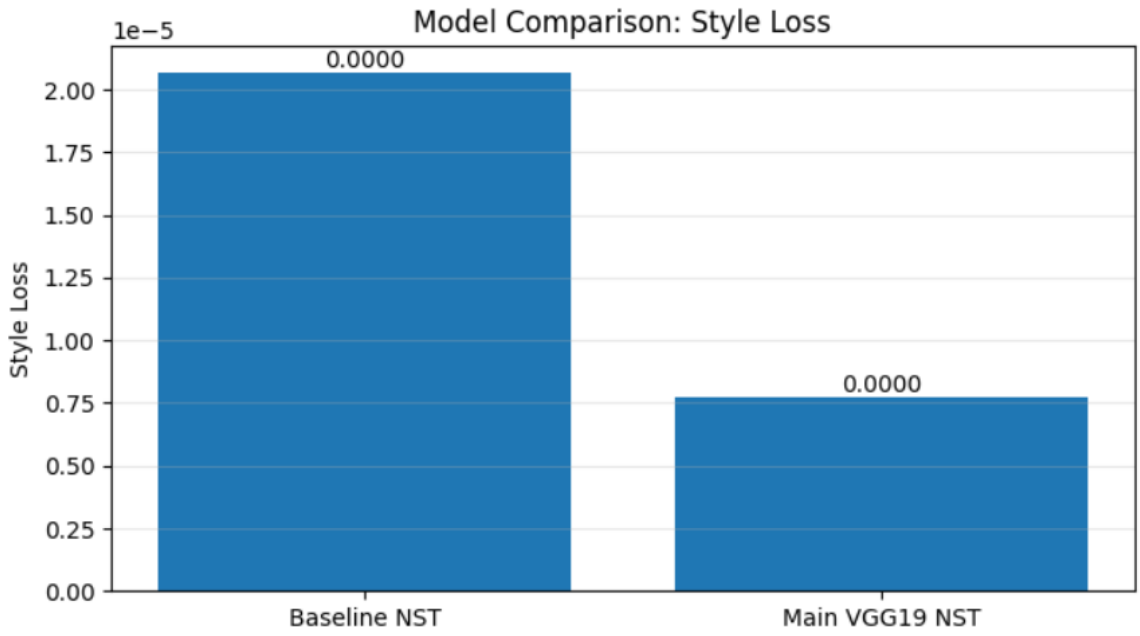
Neural style transfer can be affected by image pair, initialization, optimization settings and hardware. To make conclusions more reliable, evaluate multiple content-style pairs and repeat stochastic experiments when applicable. Report the mean and standard deviation of runtime and chosen quality measures. A statistical significance test is appropriate only when there are enough comparable independent measurements; a single image pair is not sufficient to support a broad statistical claim.

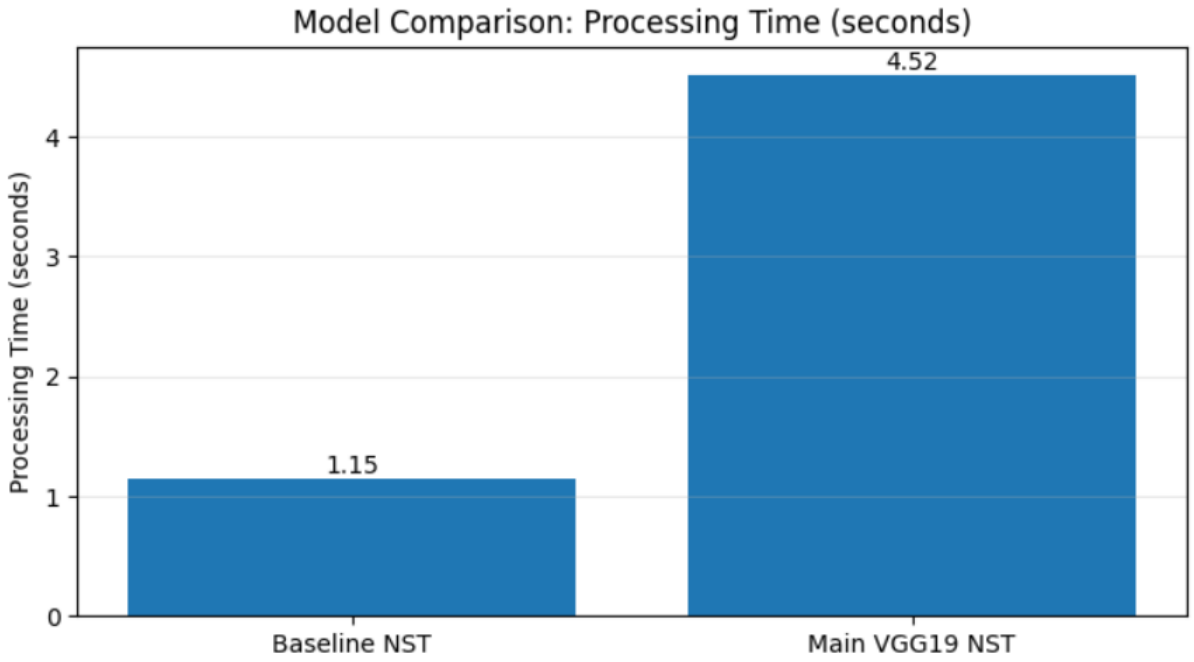
4.3 Model Complexity Analysis

VGG19 is a pretrained feature extractor with many convolutional parameters, but the project freezes these parameters and optimizes only the generated image. Consequently, the model weights are not retrained for every pair; the main cost comes from repeated forward and backward passes through the selected VGG19 layers. Larger images and more optimization steps increase computational work and memory use. The baseline configuration is expected to be less expensive

because it uses a smaller resolution and fewer steps, although actual runtime should be measured.







4.4 Visual Quality and Robustness Analysis

Robustness can be examined by testing landscapes, portraits, buildings, low-light images and images with fine details. Some style images transfer color effectively but obscure small objects. Strong style weights may increase texture transfer while changing content details; lower weights may preserve the photograph but produce a weaker artistic effect. Sharpening may emphasize edges but can also create halos or exaggerate noise, so it should remain optional.

	Model	Content Loss	Style Loss	Processing Time (seconds)
0	Baseline NST	3.389371	0.000021	1.150461
1	Main VGG19 NST	1.472636	0.000008	4.518776

4.5 Style Transfer Analysis

This enhancement is applied after style transfer. A sharpening factor of 1.0 leaves sharpness unchanged; higher values emphasize edges from baseline NST to VGG9 NST.



4.6 Discussion

The project is appropriate for an educational mini project because its pipeline is transparent and its components can be tested independently. The content loss, style loss and optimization steps provide an understandable connection between deep learning features and the final artwork. However, optimization-based NST is slower than many real-time style-transfer methods, and the output quality depends strongly on the selected style image and loss weights. Post-processing cannot recover information that was removed during stylization.

4.7 Conclusion

AI Art Studio: Photo to Painting implements an image stylization workflow using pretrained VGG19 and Neural Style Transfer. It accepts a photograph and a style reference, extracts deep features, optimizes an output image, optionally enhances sharpness and contrast, and saves the final image as PNG. The baseline and main configurations support a practical comparison of image quality, feature losses and execution time. Future improvements may include a Gradio user interface, a built-in style gallery, high-resolution processing, perceptual evaluation, and faster feed-forward style-transfer networks.

REFERENCES

1. Gatys, L. A., Ecker, A. S., and Bethge, M. (2016). Image Style Transfer Using Convolutional Neural Networks. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR).

2. Simonyan, K., and Zisserman, A. (2015). Very Deep Convolutional Networks for Large-Scale Image Recognition. International Conference on Learning Representations (ICLR).

3. PyTorch. Neural Style Transfer Tutorial. https://docs.pytorch.org/tutorials/advanced/neural_style_tutorial.html

4. Torchvision. VGG19 model documentation. https://pytorch.org/vision/stable/models/generated/torchvision.models.vgg19.html

5. PyTorch documentation. https://pytorch.org/docs/stable/index.html